GitHub Copilot: more than autocomplete

An AI assistant inside your development workflow.

Describe the goal.

Let Copilot help. Keep your judgement.

UNDERSTAND	WRITE	REVIEW	DELEGATE
Explore unfamiliar code	**Complete and change code**	**Explain and check changes**	**Assign a bounded task**

Today: choose the right interaction, give useful context, review and verify.

How it works: context in, suggestions out

The model uses learned patterns plus the information supplied for this request.

YOUR INPUT

Goal + constraints
Files + selection
History + attachments

COPILOT

Model proposes steps
Tools read / act
Results feed back

OUTPUT

Answers + suggestions
Code edits + actions
Evidence to inspect

Training is not live project knowledge

Your repo and the latest API details may need to be read. Supplying a file is not retraining the model.

Relevant context beats more context

Attach the file, screenshot or exact error. Do not assume Copilot has read the entire repository.

Same prompt, different result is possible. Inspect references; verify the output.

One product, several surfaces and capabilities

Start in the surface closest to your work. We use VS Code / Codespaces today.

IDE

Edit, chat, plan, run tools

GITHUB.COM

Repository, issue and PR work

TERMINAL

Copilot CLI agent workflows

APP / MOBILE

Coordinate or continue work

FOUR CAPABILITIES TO RECOGNISE

Copilot Chat

A conversation to understand code and direct work.

PR summaries

Draft a description of a pull request's changes.

Code review

Find potential issues and suggest improvements.

Coding agent

Delegate a task. Review its changes in a pull request.

A summary explains a diff. A review critiques it. A coding agent changes code.

DO NOT MIX UP THE SELECTORS

Surface, target, mode and model are different

Changing one does not necessarily change the others.

SURFACE	Where do I interact?	VS Code, GitHub, CLI
TARGET	Which runtime / environment owns this session?	Local, Copilot, Cloud
MODE / ROLE	What kind of help do I want?	Ask, Plan, Agent
MODEL	Which AI engine reasons over the request?	Balance quality, speed and usage

Local

Built-in editor harness.
Legacy, editor-tied sessions.

Copilot

Copilot SDK + Agent Host.
Shared agent runtime.

Cloud

Managed cloud execution.
Separate workspace.

Local does not mean offline. Target is not model. Controls vary by target and version.

INSIDE THE EDITOR

Small change? Stay close to the code.

Completion, inline chat and next edit suggestions solve different problems.

Ghost text

Finish what I am typing

```
// Count orders by branch
async countByBranch(...)
  // suggested code
```

Tab: accept
Esc: dismiss

Inline chat

Change this selected code

```
SELECT METHOD
Add a short JSDoc comment
```

Ctrl+I / Cmd+I
Review the proposed diff

Next edit suggestions

Suggest the related next edit

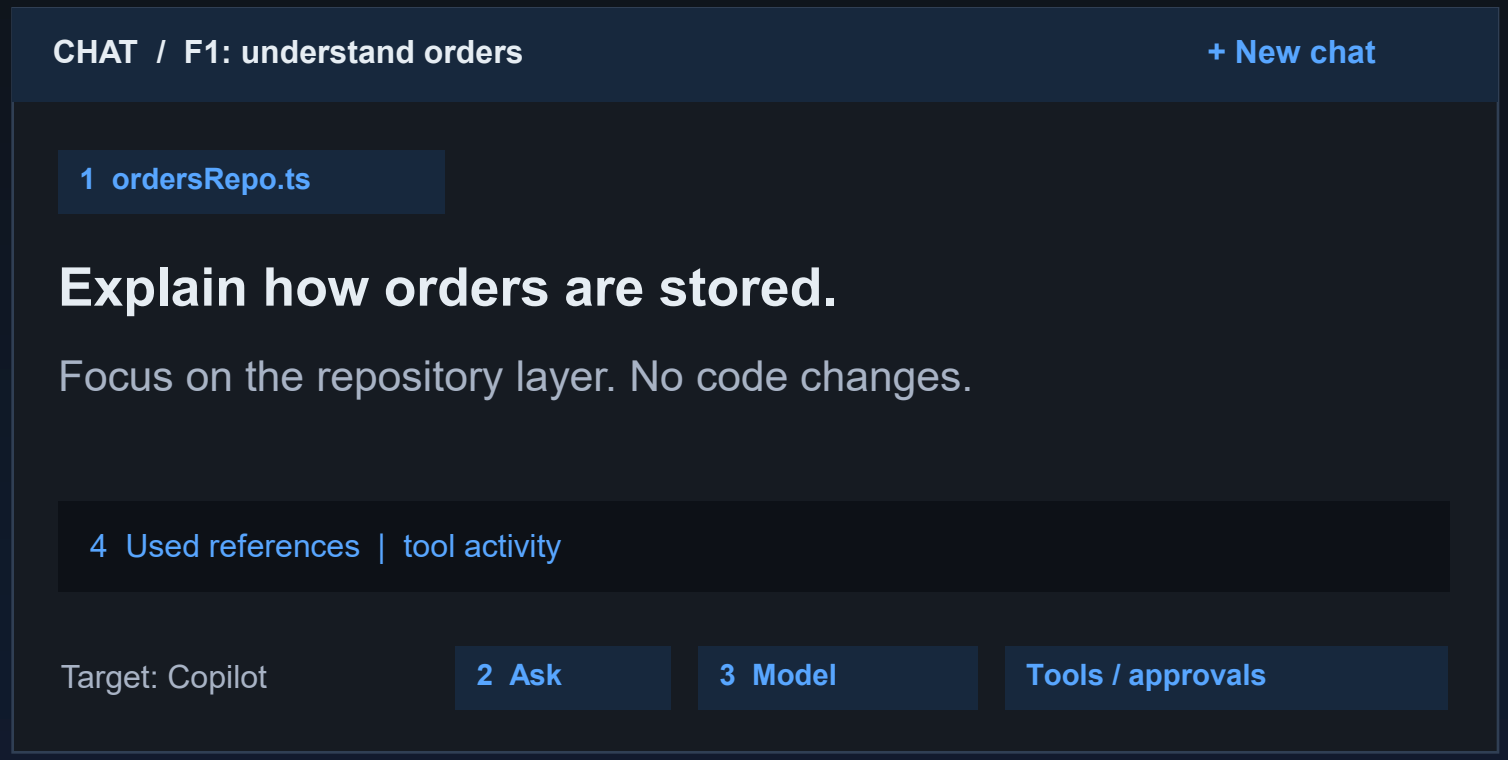
```
Change a parameter
Review the next call site
```

Tab: navigate
Tab again: accept

Need broader context, several files or tools? Move to the Chat panel.

Know your control desk before you delegate

Open Chat from the VS Code title bar. The diagram is schematic, not a UI screenshot.



1 Attach relevant context

Use #, Add Context or drag a file.

2 Choose the mode / role

Ask, Plan or Agent for this task.

3 Choose an available model

Keep the workshop default today.

4 Inspect what was used

Read references and tool results.

`/explain` `/fix` `/tests` Type / for available shortcuts. Commands are not modes.

CHOOSE THE INTERACTION

Ask, Plan or Agent? Match the job.

Choose deliberately. You do not need an agent to answer every question.

Ask

UNDERSTAND

Explore code and get explanations without code changes.

Where should this endpoint go?

Plan

THINK FIRST

Research the task.
Clarify the scope.
Review the proposed steps.

Plan a cart from this design. Do not build it.

Agent

BUILD + CHECK

Find files, edit code and use allowed tools; iterate on results.

Expose countByBranch as a GET endpoint with Swagger docs.

F1 uses all three: Ask to understand, Agent to build, Plan the cart without building it.

A better prompt removes guesswork

Good prompting supplies the information needed to produce a useful answer.

TOO VAGUE

explain this project

Audience? Goal? Focus? Format?

MORE USEFUL

I'm a data engineer, new to this repo.
Help me understand the project.
Explain what it does, its stack, how to run and test it.
Focus on the API. Answer in at most 10 bullets.

CONTEXT	INTENT	CLARITY	SPECIFICITY
Role + starting point	**The outcome you want**	**A direct, concrete task**	**Scope + output limit**

Judge usefulness, not length. For build tasks, add examples, constraints and finish checks.

Improve the loop, not just the wording

Treat the response as a draft to inspect, not a result you must accept.

1 Describe

Goal + relevant context

2 Inspect

What is missing or wrong?

3 Refine

Add precise feedback

4 Verify

Check the outcome

Instead of "try again"...

"Focus on the order model, repository and route. List the relevant files and explain the branch filter. Do not write code."

Missing evidence?

Attach the exact file or error.

Task too large?

Split it; plan before building.

Different task?

Start a fresh chat; carry key decisions.

You stay in control: before, during and after

A confident answer can still be wrong. Instructions steer; permissions limit actions.

BEFORE

Set the goal and boundaries.

Use approved tools and synthetic data; keep secrets out.

DURING

Inspect tool calls and approvals.

Stop or redirect drift. Review what each command will do.

AFTER

Read every changed-file diff.

Keep / Undo deliberately. Run checks or try the endpoint.

Done = requirement met + changes reviewed + behaviour verified

Undoing files or restoring a checkpoint may not undo deployments or external actions.

YOUR WORKSHOP APPLICATION

Meet OctoCAT Supply

A fictional smart-cat-products business, with products, branches, orders and supplier deliveries.





The business question

"How many orders belong to my branch?"

WHAT THE APP DOES

Browse products and administer the catalog.
Explore orders and deliveries through the API / Swagger.

Website
React + TypeScript

→

API
Express + Swagger

→

Database
SQLite

SAME APP, YOUR OWN LENS

Frontend
Catalog UX, types and API contracts

Backend
Routes, repositories and validation

Data
Relationships, queries and migrations

Platform
Run, test and deploy consistently

NOW TRY IT | 30-MINUTE HANDS-ON

Lab F1: your first ticket

"Show how many orders each branch has."

1	2	3	4
8 MIN	5 MIN	10 MIN	7 MIN
Ask	**Completion + inline chat**	**Agent**	**Plan**
Compare prompts. Find the relevant files.	Add countByBranch. Add a JSDoc comment.	Build GET + Swagger. Try branchId = 1.	Attach the cart design. Read the plan; do not build.

DONE WHEN

Swagger returns a branch order count, and you have reviewed a cart plan.

Open labs/F1-prompting-fundamentals.md | Describe. Review. Verify.